

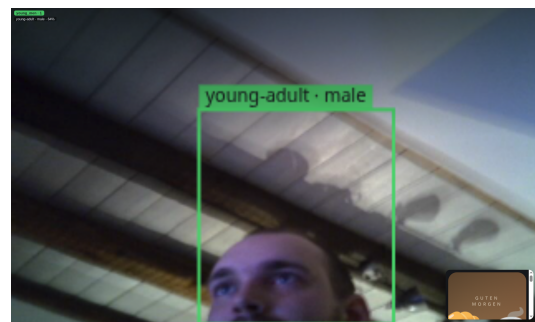
Einführung

Beruflich entwickle ich seit einiger Zeit an einem Digital-Signage-System (digitale Werbebeschilderung), welches Inhalte über einen zeitgesteuerten Ablaufplan ausspielt. Offensichtlich ist damit ohne das manuelle Eingreifen der Betreiber keine kontextsensitive Auswahl möglich und es geht Werbepotenzial verloren. Daraus ergibt sich für mich die unmittelbare praktische Frage, ob sich diese statischen Kanäle durch eine KI-gestützte, kontextabhängige Auswahl ersetzen lassen und welche Architektur die dafür nötigen KI-Funktionen am sinnvollsten umsetzt.

Vor diesem Hintergrund habe ich im Rahmen dieses Projekts einen Prototyp für *kontextsensitive Digital Signage* entwickelt, der drei architektonische Varianten der Inhaltsauswahl direkt vergleichbar macht: *Edge*-, *Hybrid*- und *Cloud-Ansatz*. Auf den nächsten Seiten werden diese diskutiert und entlang von vier Kriterien bewertet: Performance (Latenz je Auswahl), Datenschutz (verlässt das Kamerabild das Gerät?), Korrektheit (wählt das System den situativ passenden Inhalt?) und Kosten (Cloud-Aufrufe und Bandbreite). Abbildung 1 zeigt den Prototyp im Betrieb.



(a) Ausgespielter Werbeinhalt.



(b) On-device Zielgruppenerkennung.

Abbildung 1: Der Prototyp im Betrieb.

Systemüberblick

Das System besteht aus zwei eigenständigen Anwendungen. Die erste läuft auf dem Board und ist insb. für das Sammeln von Bildinformationen und das Anzeigen der Werbeinhalte verantwortlich. Die zweite ist eine **FastAPI**-Anwendung, die als CMS die Inhalte verwaltet. Wo die Auswahl der anzuzeigenden Inhalte stattfindet, ist die zentrale Frage der Arbeit und unterscheidet sich je nach Variante.

Als Hardware der Board-Anwendung dient der **Arduino UNO Q**,¹ ein Dual-Brain-Board aus einem Linux-MPU und einem STM32-MCU. Intern ist die Board-Anwendung als *Strategy-Slot-Pipeline* aufgebaut (Abbildung 2). Ein **Trigger** stößt pro Zyklus genau einen Durchlauf an. Anschließend laufen drei Stufen nacheinander: Die Stufe **Audience** klassifiziert on-device die Zielgruppe, der **Selector** wählt auf Basis dieses Kontexts einen Inhalt, und der **Sink** schickt das Ergebnis an die Webapp. Jede Stufe ist ein austauschbarer Slot mit einheitlicher Schnittstelle.

¹Arduino. *Arduino UNO Q und App Lab*. <https://docs.arduino.cc/software/app-lab/>. 2025.

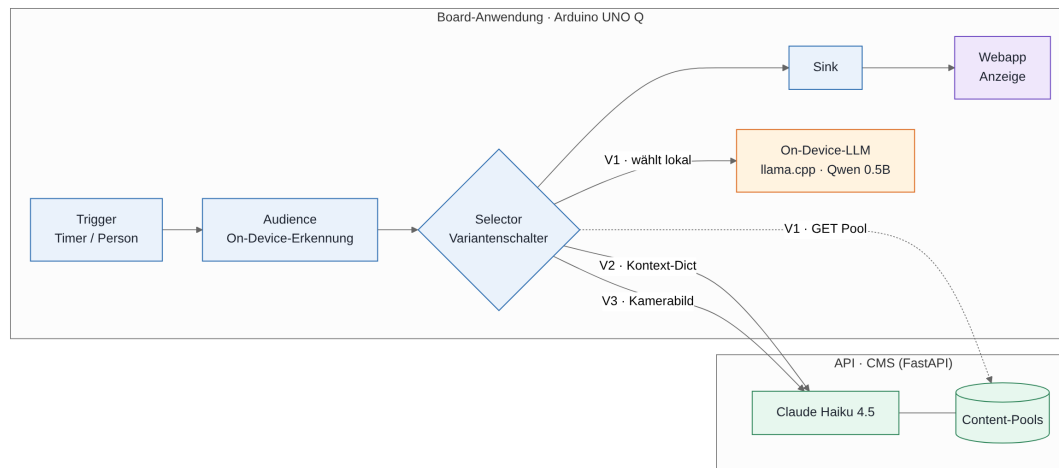


Abbildung 2: Die Strategy-Slot-Pipeline: vier austauschbare Stufen.

Content-Management-System

In der Praxis verwaltet ein Content-Management-System die anzuzeigenden Inhalte und spielt sie über zeitgesteuerte Kanäle. Für die KI-Auswahl vollziehe ich einen Paradigmenwechsel: An die Stelle fester Kanäle treten *Pools*, also Kandidatenmengen, aus denen pro Zyklus ein darzustellender Inhalt gewählt wird.

Umgesetzt ist das hier als die erwähnte API-Anwendung als schlanker Platzhalter für mein Produktiv-System. Die Inhalte liegen als HTML-Dateien auf der Platte und in der Datenbank stehen nur die Metadaten. Die API drei Endpunkte bereit: `GET /pools/{id}` liefert den kompletten Pool, während `choose-by-context` und `choose-by-img` die Auswahl selbst treffen und nur einen Inhalt zurück senden. Für diese Auswahl in der Cloud nutze ich Claude Haiku 4.5.²

Zielgruppenerkennung und lokale LLMs

Damit die Audience-Stufe ein belastbares Signal liefert, habe ich eine kamerabasierte Erkennung implementiert, die ausschließlich on-device arbeitet. Der Ablauf je Frame umfasst drei Schritte: Zunächst dekodiert `cv2.imdecode` das JPEG. Anschließend detektiert YuNet³ die Gesichter und liefert Crops. Schließlich schätzt ein selbst trainiertes Modell pro Gesicht das Alter und das Geschlecht. Das verwendete Modell habe ich mithilfe von FairFace⁴ trainiert und nach ONNX exportiert. Die Einzelergebnisse aggregiere ich zu einer szenenweiten `target_group` (bspw. `families_with_children` oder `seniors`).

Die Bildererkennung läuft auf dem UNO Q überraschend gut. YuNet und das MobileNetV3-small-Netzwerk sind kompakte für den Edge-Einsatz entworfene Technologien, sodass die Zielgruppenerkennung in wenigen Millisekunden ausgeführt werden kann.

Ein On-Device-LLM läuft dagegen nicht schnell genug. Hierzu habe ich mit einem llama.cpp-Server⁵ und dem eher kleinen Modell Qwen2.5-0.5B experimentiert. Gemes-

²Anthropic. *Claude – Modelle und API*. <https://docs.anthropic.com>. 2025.

³OpenCV. *OpenCV Zoo: YuNet Face Detection*. https://github.com/opencv/opencv_zoo. 2023.

⁴Kimmo Kärkkäinen und Jungseock Joo. *FairFace: Face Attribute Dataset for Balanced Race, Gender, and Age*. <https://github.com/joojs/fairface>. 2021.

⁵Georgi Gerganov und llama.cpp contributors. *llama.cpp: LLM inference in C/C++*. <https://>

sen auf dem Board erreicht das Modell nur etwa 6 bis 7,5 Tokens pro Sekunde bei der Prompt-Auswertung und magere 1,4 bis 1,8 Tokens pro Sekunde bei der Generierung (rund 600 ms je Token). Eine einzelne Auswahl dauert so leicht ein bis zwei Minuten.

Variante 1: Edge-only

In der ersten Variante wählt das Board den Inhalt selbst, mithilfe eines On-Device-LLM. Es ruft dazu über `GET /pools/{id}` den gesamten Kandidaten-Pool von der API ab. Den lokal verwendeten Prompt baue ich gezielt in zwei Teilen auf: zuerst einen stabilen Teil (System-Anweisung und Kandidatenliste aus ID, Name und einer Inhalts-Beschreibung ohne der eigentlichen HTML) und den kleinen, wechselnden Kontext. Dadurch kann der Runner den Key-Value-Cache des LLM über mehrere Aufrufe hinweg wiederverwenden. Als Antwort verlange ich ausschließlich die Integer-ID des darzustellenden Inhalts.

Der größte Aufwand steckte hier in der Optimierung der LLM-Nutzung. Anfangs benötigte eine einzelne Auswahl rund 230 Sekunden. Über mehrere Schritte habe ich die Latenz deutlich senken können, indem ich das HTML aus dem Prompt entfernt, von einer JSON- auf eine reine ID-Antwort umgestellt und das erwähnte Caching aktiviert habe. Schlussendlich dauert die erste Auswahl noch rund 96 Sekunden und jede folgende nur noch etwa 20 Sekunden. Die Auswahlqualität des 0,5B-Modells bleibt jedoch grob. In einem Test mit fünf klar gelagerten Kontexten traf es drei korrekt. Ein noch kleineres Modell würde darunter fallen, weshalb dieses Modell die sinnvolle Untergrenze markiert.

Die Edge-only Variante erreicht damit die beste Bewertung bei Datenschutz und Kosten (keine Cloud-Aufrufe), erkauft das aber mit der im Vergleich schwächsten Korrektheit und der mit Abstand schlechtesten Performance.

Variante 2: Hybrid

Die zweite Variante erkennt die Zielgruppe weiterhin lokal, überlässt die Auswahl aber der Cloud: Das Board sendet nur ein abstraktes Kontext-Dict an `POST /pools/{id}/choose-by-context` und Claude wählt daraus den passenden Inhalt aus dem Pool.

Der eigentliche Vorzug dieser Variante liegt in der Flexibilität des Übergabeformats. Das Kontext-Dict ist offen geformt, sodass sich beliebige weitere Signale (erkannte Zielgruppe, Tageszeit, künftig auch Sensordaten) ohne jeden Schnittstellenbruch einhängen lassen und direkt in den Prompt fließen. Die Architektur skaliert also mit der Menge des verfügbaren Kontexts. Ein nennenswertes Problem ergab sich dabei nicht beim Cloud-Aufruf selbst, sondern in seiner Voraussetzung. Dieser Ansatz ist immer nur so gut wie das lokale Zielgruppenerkennung. Genau das hat die oben beschriebene On-Device-Zielgruppenerkennung überhaupt erst erzwungen.

Auf die vier Kriterien bezogen liegt diese Variante durchgängig im guten Mittelfeld. So liefert sie starken Datenschutz (das Bild wird nur lokal verarbeitet), eine gute Korrektheit dank Cloud-LLM sowie moderate Performance und Kosten, da je Auswahl nur ein kompakter Text-Prompt in die Cloud geht.

github.com/ggml-org/llama.cpp. 2024.

Variante 3: Cloud-only

In der dritten Variante lädt das Board den rohen Kamera-Frame zu `POST /pools/{id}/choose-by-img` hoch und Claude übernimmt dann Bilderkennung sowie Auswahl in einem Schritt. Anders als die Edge- und die Hybrid-Variante führt diese Version keine Zielgruppenerkennung auf dem Board aus.

Im Test zeigte sich, dass das Rohbild mehr Information als das beabsichtigte Personen-Signal beinhaltet, sodass Szenen- und Hintergrund-Hinweise die Auswahl überlagern können. Hinzu kommt der prinzipielle Nachteil, dass das Rohbild das Gerät verlässt, was Datenschutz, Bandbreite und Netzlatenz belastet.

Auf die vier Kriterien bezogen kehrt diese Variante also das Bild von der ersten Lösungsstrategie um. Sie liefert verlässlich die beste Korrektheit, aber die schwächsten Werte bei Datenschutz (das Rohbild verlässt das Gerät), bei den Kosten (Vision-Tokens und Bildbandbreite je Auswahl) und eine netzabhängige Performance.

Vergleich, Empfehlung und Ausblick

Variante	Performance	Datenschutz	Korrektheit	Kosten
Edge	schwach	sehr gut	schwach	sehr gut
Hybrid	mittel	gut	gut	gut
Cloud	netzabhängig	schwach	sehr gut	schwach

Tabelle 1: Vergleich der drei Varianten entlang der vier Bewertungskriterien.

Tabelle 1 fasst die Bewertungen zusammen. Keine der vorgeschlagenen Varianten führt auf allen vier Kriterien. Edge-only punktet bei Datenschutz und Kosten, ist aber zu langsam und ungenau. Cloud-only liefert die beste Korrektheit, gibt dafür jedoch das Kamerabild aus der Hand und verursacht die höchsten Kosten. Trotzdem verbindet der Hybrid-Ansatz die jeweiligen Stärken der anderen beiden Modelle und umgeht jeweils ihre Schwachstellen. Die Bilderkennung läuft lokal schnell und bleibt deshalb auf dem Board, sodass das Kamerabild das Gerät nie verlässt. Die für das kleine On-Device-LLM zu anspruchsvolle Auswahl wandert dagegen in die Cloud.